CSE221: Algorithms

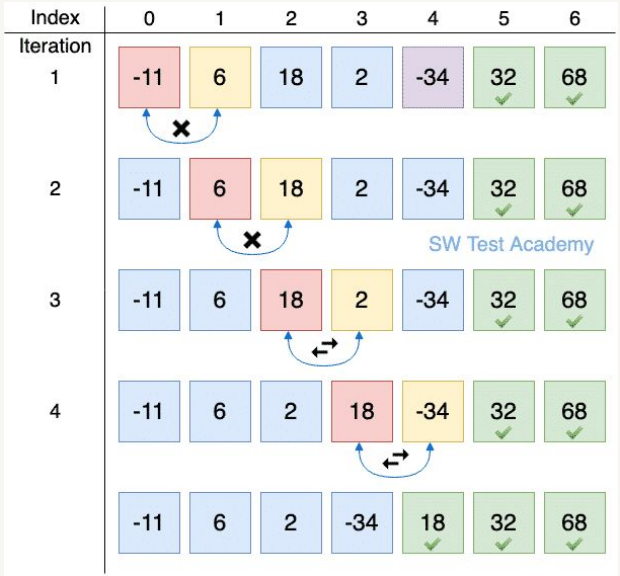
Comparison Based Sorting Algorithms

Prepared by: A B M Muntasir Rahman [MUNR]

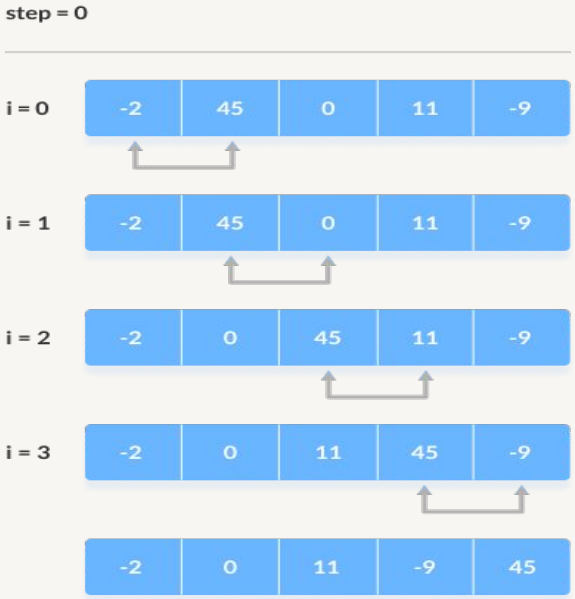


Bubble Sort

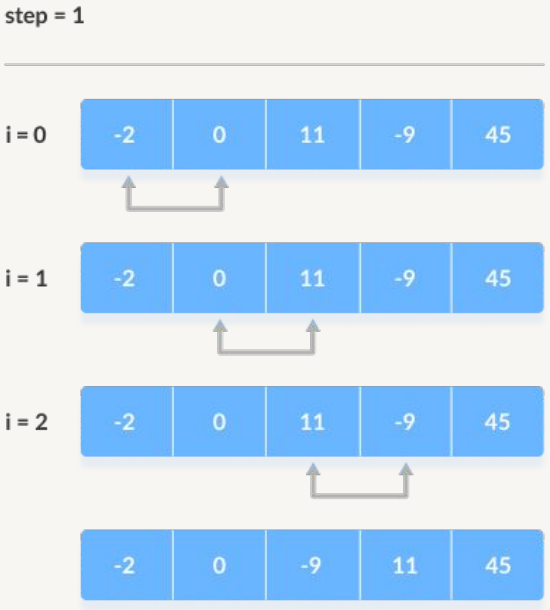
- A simple sorting algorithm that works by comparing two adjacent elements and swaps them until they are in correct order.
- Just like air bubbles in waters rises up to the surface, at the end of each pass in Bubble Sort highest element moves to the end (while sorting in ascending order) or lowest element moves to the end (while sorting in descending order).



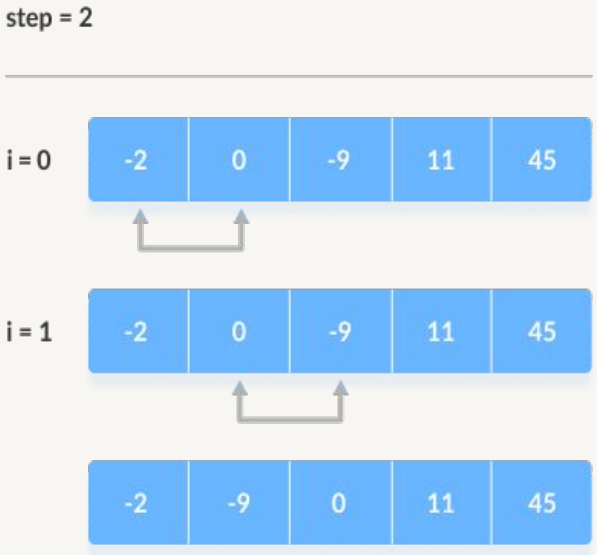
Bubble Sort (Simulation)



Bubble Sort (Simulation)



Bubble Sort (Simulation)



Bubble Sort (Pseudocode)

```
bubbleSort(array)
  for i <- 0 to sizeOfArray - 1
    for j <- 0 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
  end bubbleSort
```

Optimized Bubble Sort (Pseudocode)

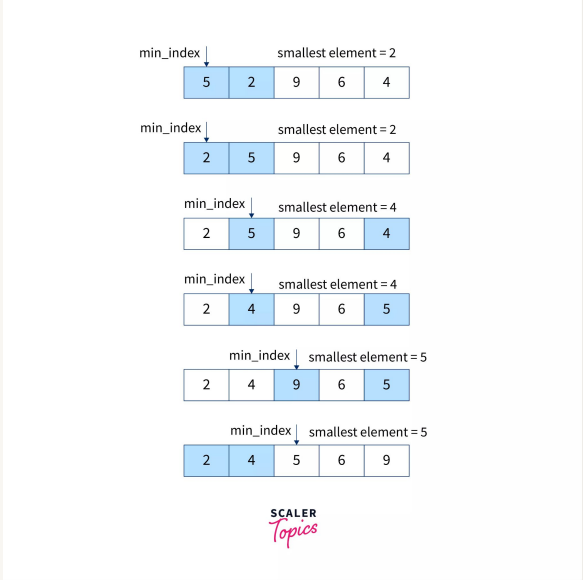
```
bubbleSort(array)
  for i <- 0 to sizeOfArray - 1
    swapped <- false
    for j <- 0 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
        swapped <- true
    if swapped equals false
      break
  end bubbleSort
```

Bubble Sort (Properties)

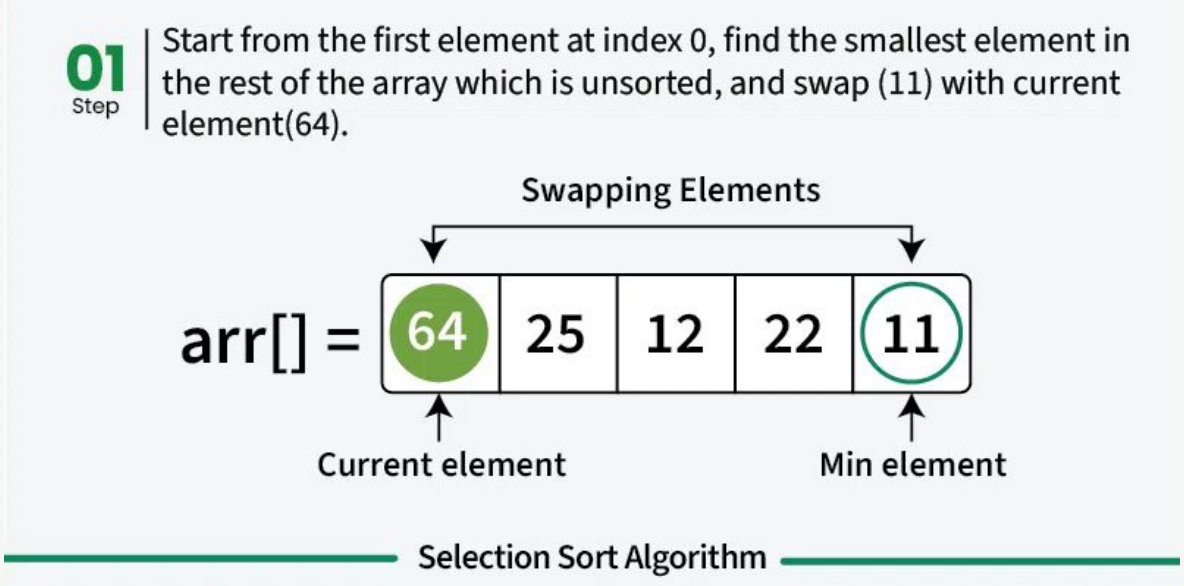
- **Time Complexity:**
  - Best Case: O(n) when array is sorted already.
  - Average Case: $O(n^2)$ when array is in random order.
  - Worst Case: $O(n^2)$ when array is in reverse order.
- **Space Complexity:**
  - Best/Average/Worst case: O(1).
- **Stable:** Yes.
- **In-place:** Yes.
- Suitable for smaller arrays. Unsuitable for larger arrays and when lesser swapping is preferred.

Selection Sort

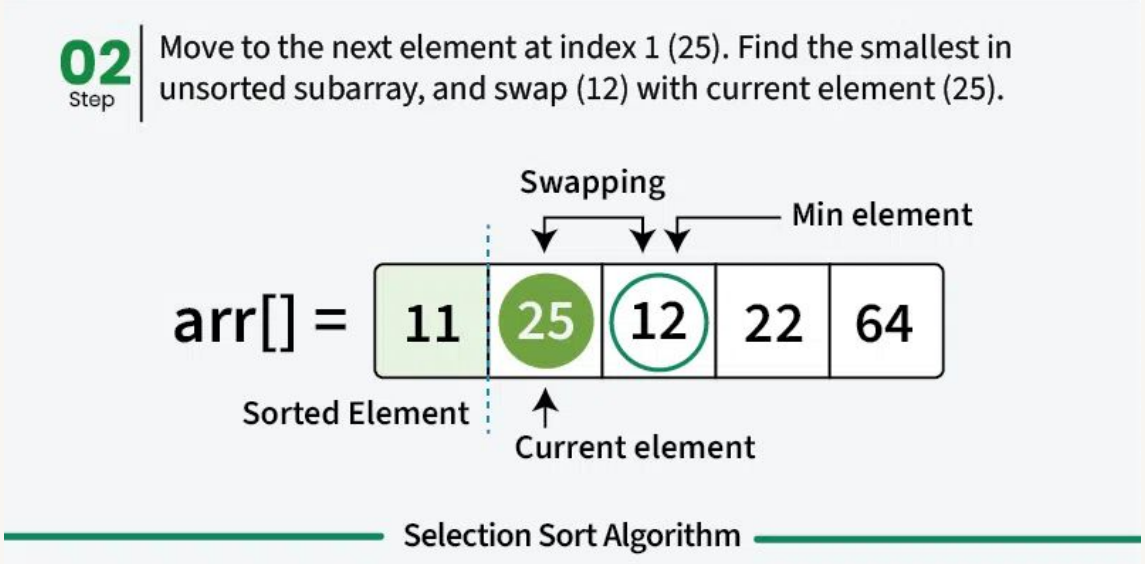
- A simple sorting algorithm that selects the lowest value in an array and moves it to the front of the array.
- The basic logic is to select the smallest element from the unsorted part and swap it with the min_index element.



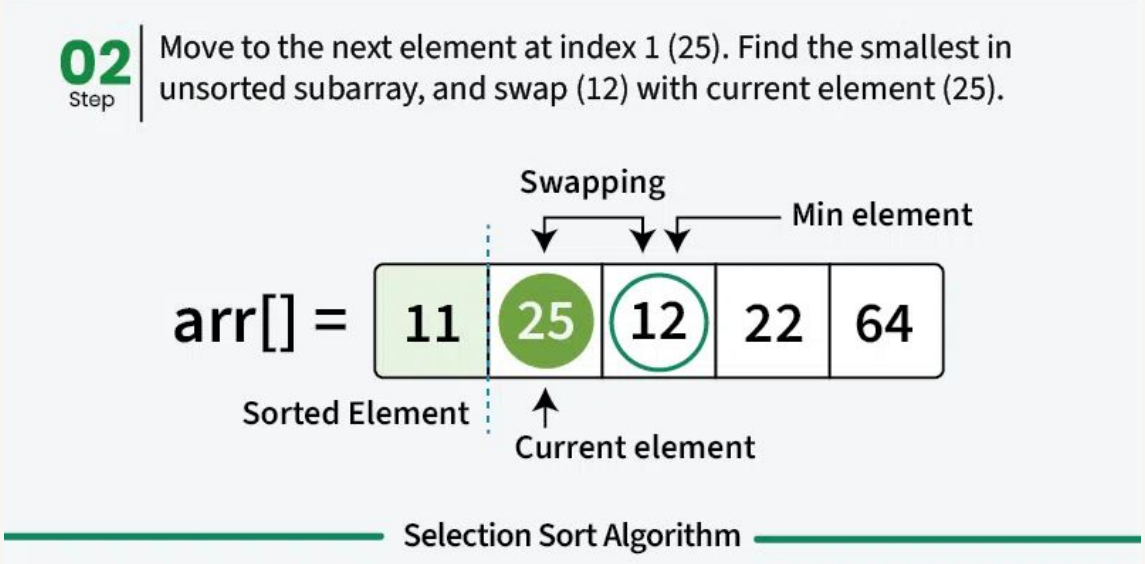
Selection Sort (Simulation)



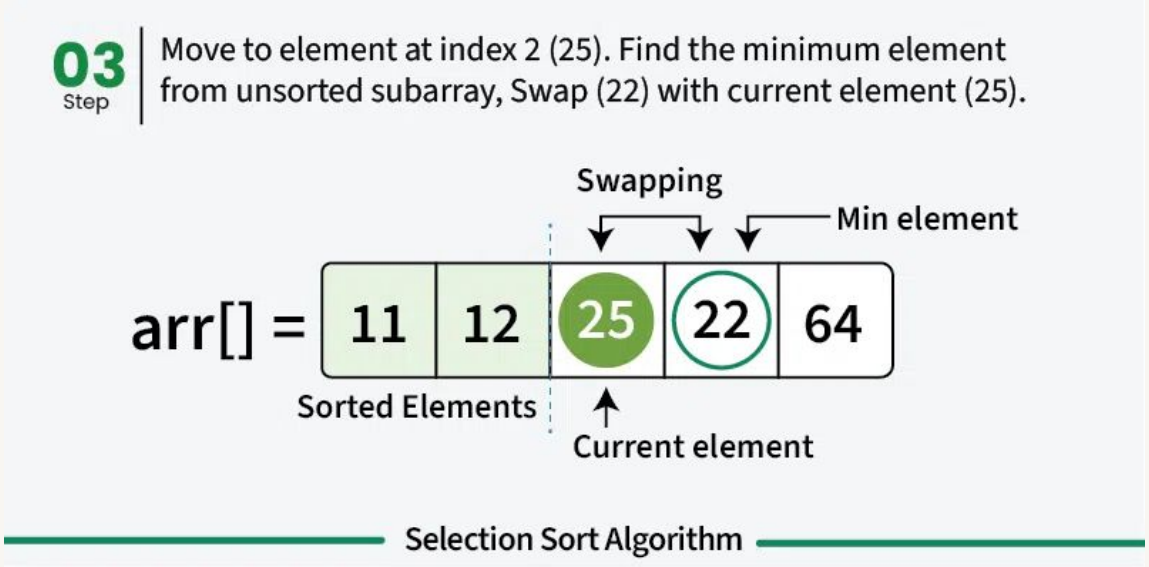
Selection Sort (Simulation)



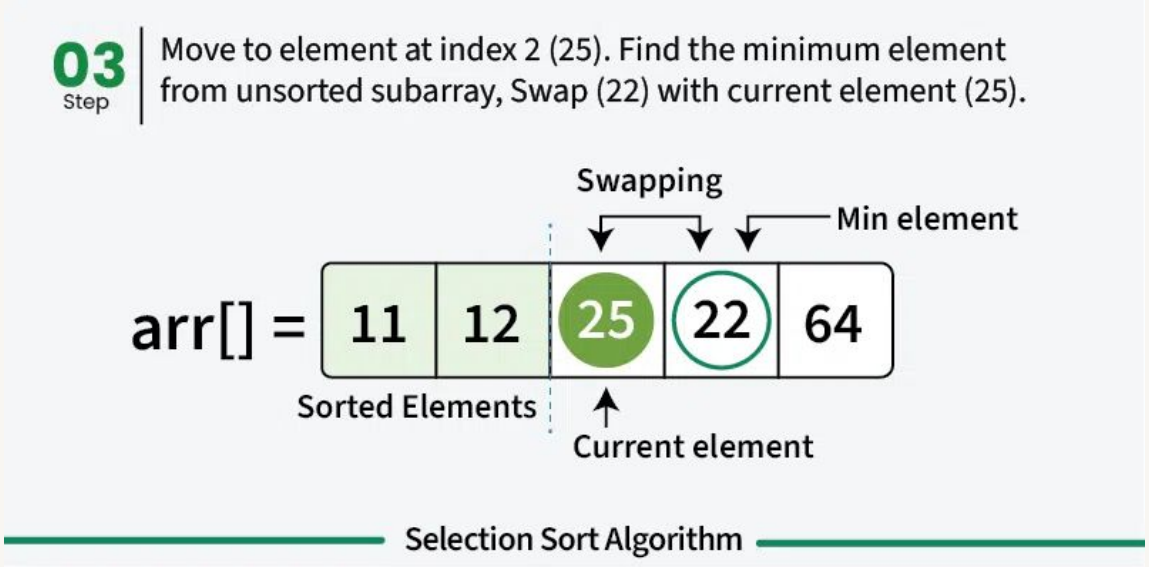
Selection Sort (Simulation)



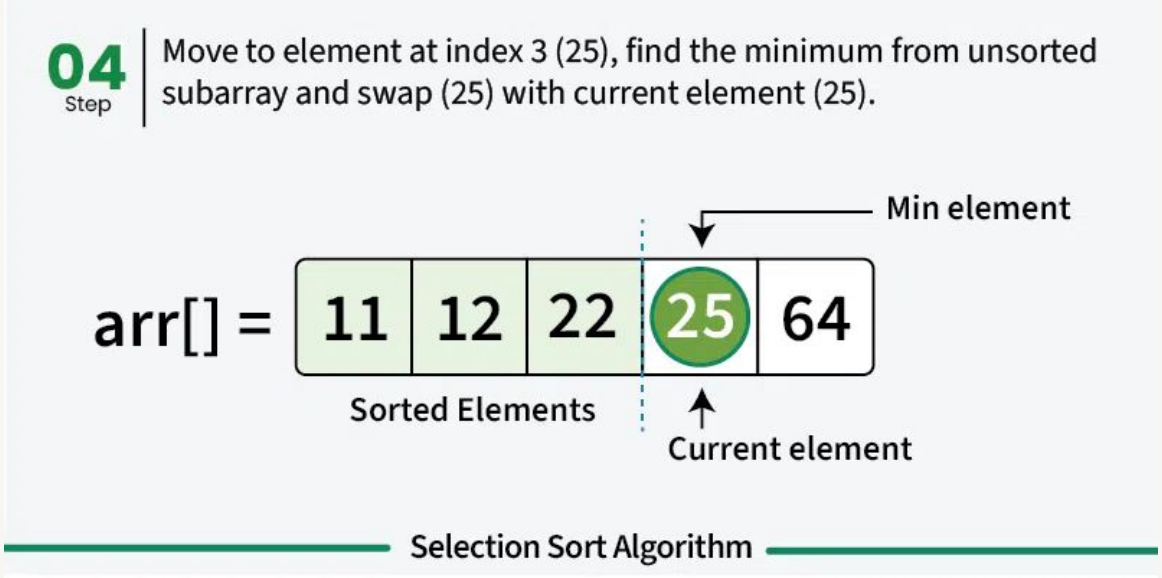
Selection Sort (Simulation)



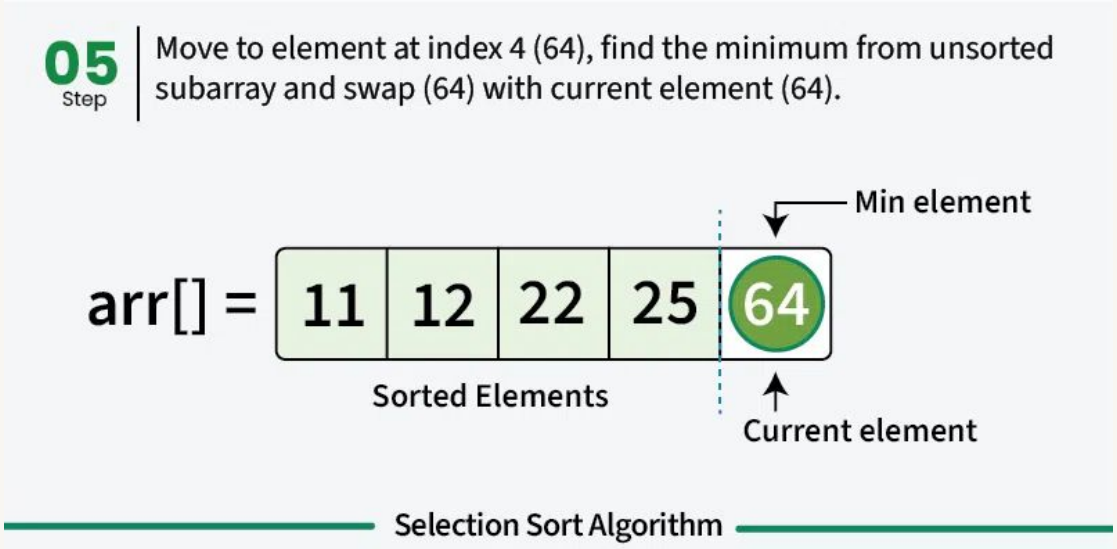
Selection Sort (Simulation)



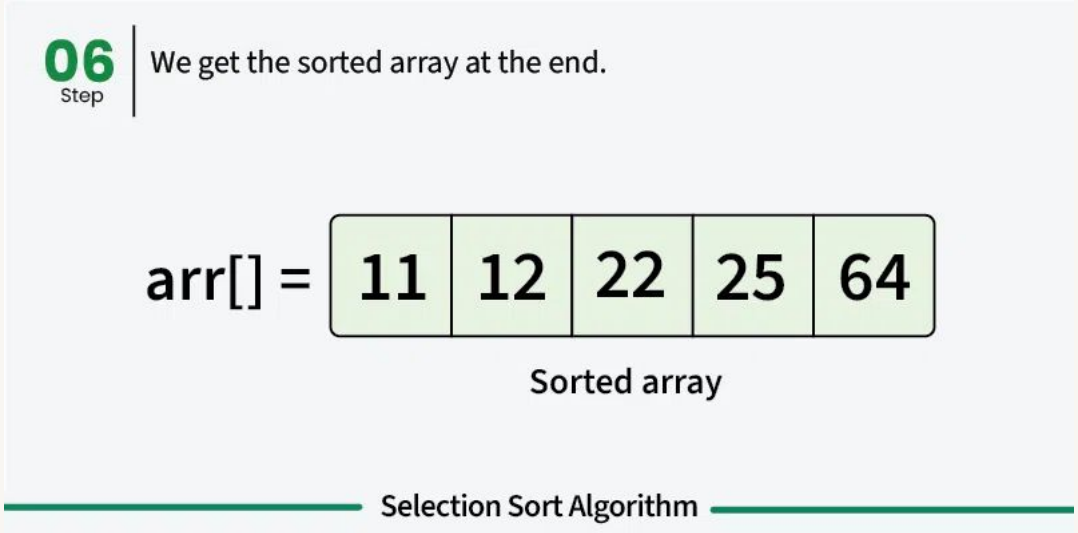
Selection Sort (Simulation)



Selection Sort (Simulation)



Selection Sort (Simulation)



Selection Sort (Pseudocode)

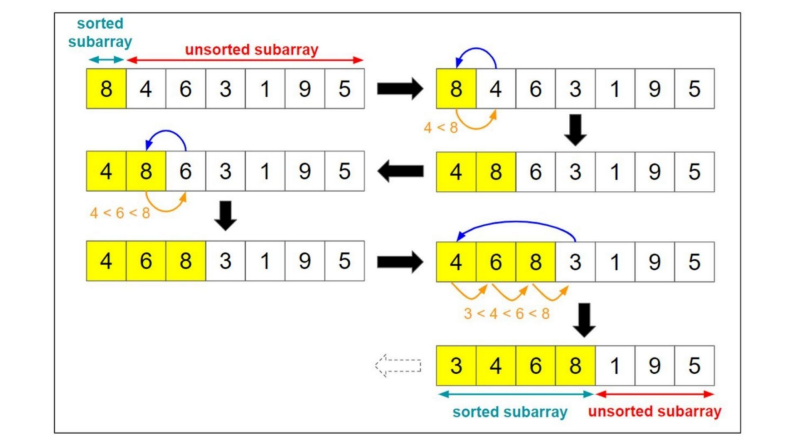
```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
  end selectionSort
```

Selection Sort (Properties)

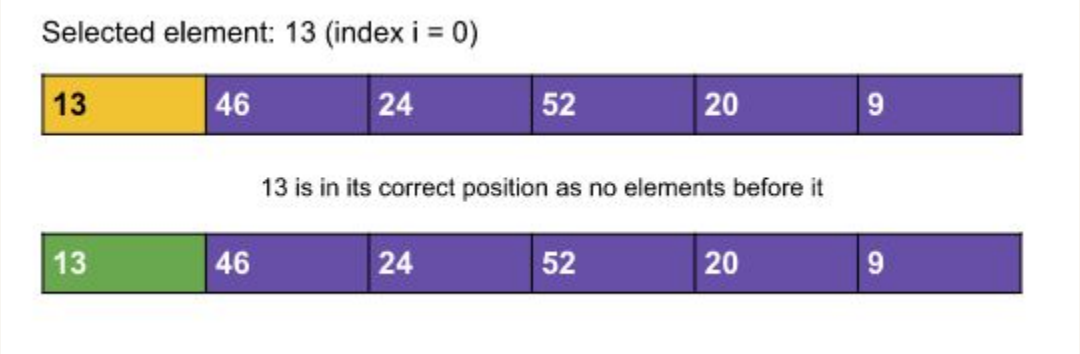
- **Time Complexity:**
  - Best Case: $O(n^2)$ when array is sorted already.
  - Average Case: $O(n^2)$ when array is in random order.
  - Worst Case: $O(n^2)$ when array is in reverse order.
- **Space Complexity:**
  - Best/Average/Worst case: O(1).
- **Stable:** No.
- **In-place:** Yes.
- Suitable for smaller arrays and when lesser swapping is preferred. Unsuitable for larger arrays.

Insertion Sort

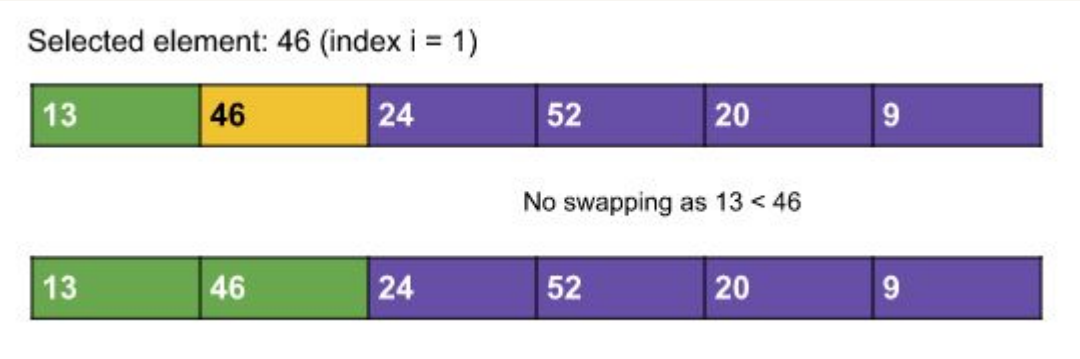
- A simple sorting algorithm that works by iteratively inserting each element of an unsorted list into it's correct position in sorted list by comparing.



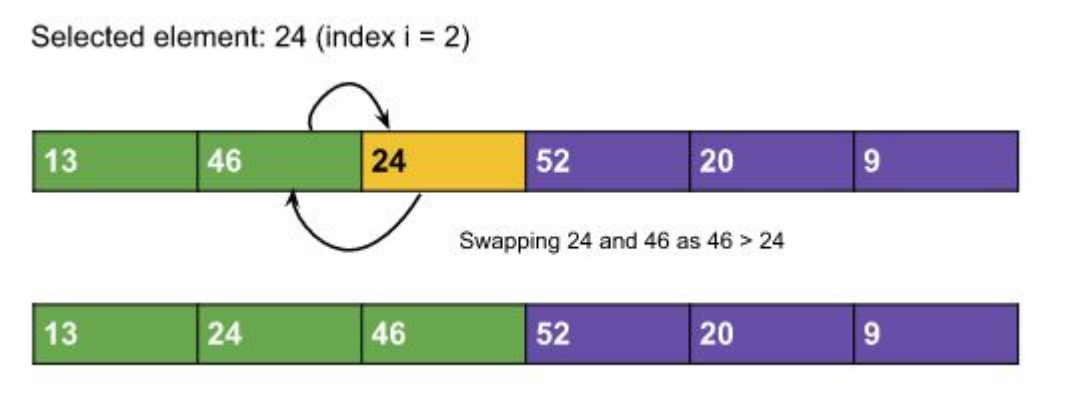
Insertion Sort (Simulation)



Insertion Sort (Simulation)



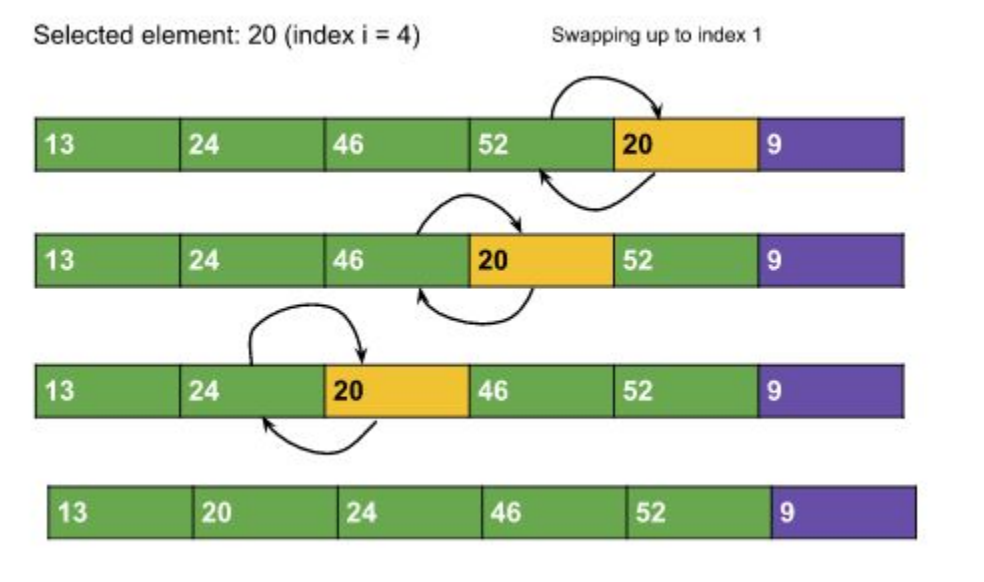
Insertion Sort (Simulation)



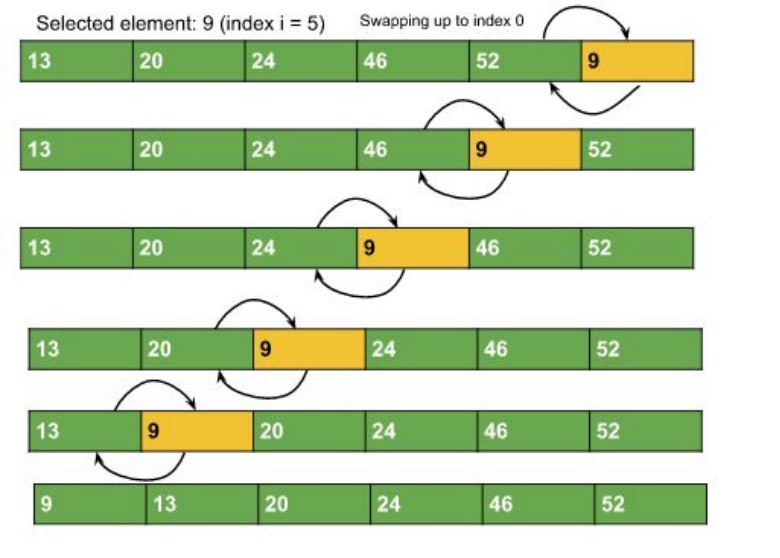
Insertion Sort (Simulation)



Insertion Sort (Simulation)



Insertion Sort (Simulation)



Insertion Sort (Pseudocode)

```
insertionSort(array)
  mark first element as sorted
  for each unsorted element X
    'extract' the element X
    for j <- lastSortedIndex down to 0
      if current element j > X
        move sorted element to the right by 1
    break loop and insert X here
  end insertionSort
```

Insertion Sort (Properties)

- **Time Complexity:**
  - Best Case: O(n) when array is sorted already.
  - Average Case: $O(n^2)$ when array is in random order.
  - Worst Case: $O(n^2)$ when array is in reverse order.
- **Space Complexity:**
  - Best/Average/Worst case: O(1).
- **Stable:** Yes.
- **In-place:** Yes.
- Suitable for smaller arrays and when a few elements needs to be sorted. Unsuitable for larger arrays.